

Лабораторная работа №5: Сети Петри. Обедающие философы

David Wanchinga

2026-09-02

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Реализация сети Петри	7
3.2	Модуль DiningPhilosophers	7
3.3	Результаты моделирования	8
4	Выводы	10

Список иллюстраций

3.1	Код структуры PetriNet	7
3.2	Код модуля DiningPhilosophers	8
3.3	Данные моделирования	8
3.4	Результаты моделирования	8
3.5	Сравнение данных	9

Список таблиц

1 Цель работы

Изучить аппарат сетей Петри на примере задачи «Обедающие философы», исследовать проблему взаимной блокировки (deadlock) и способы её предотвращения.

2 Задание

1. Построить сеть Петри для задачи «Обедающие философы».
2. Обнаружить состояние взаимной блокировки (deadlock).
3. Модифицировать сеть с помощью арбитра для предотвращения deadlock.
4. Провести имитационное моделирование и сравнить результаты.

3 Выполнение лабораторной работы

3.1 Реализация сети Петри

Была реализована структура `PetriNet` для моделирования сети Петри.

```
anch@MANKHINGA:~/work/study/2026-1/2026-1==study-simulation-modeling/labs/lab05/project$ cat src/DiningPhilosophers_noplot.jl | head -20
module DiningPhilosophers

using OrdinaryDiffEq
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock

struct PetriNet
    n_places :: Int
    n_transitions :: Int
    incidence :: Matrix{Int}
    place_names :: Vector{Symbol}
    transition_names :: Vector{Symbol}
end
```

Рисунок 3.1: Код структуры `PetriNet`

3.2 Модуль `DiningPhilosophers`

Создан модуль для построения классической сети и сети с арбитром.

```
wanch@WANCHINGA:~/work/study/2026-1/2026-1==study-simulation-modeling/labs/lab05/project$ cat src/DiningPhilosophers.noplots.jl | head -20
module DiningPhilosophers

using OrdinaryDiffEq
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock

struct PetriNet
    n_places :: Int
    n_transitions :: Int
    incidence :: Matrix{Int}
    place_names :: Vector{Symbol}
    transition_names :: Vector{Symbol}
end
```

Рисунок 3.2: Код модуля DiningPhilosophers

3.3 Результаты моделирования

3.3.1 Данные моделирования

```
wanch@WANCHINGA:~/work/study/2026-1/2026-1==study-simulation-modeling/labs/lab05/project$ cat data/dining_classic.csv | head -10
time,Think_1,Think_2,Think_3,Think_4,Think_5,Hungry_1,Hungry_2,Hungry_3,Hungry_4,Hungry_5,Eat_1,Eat_2,Eat_3,Eat_4,Eat_5,Fork_1,Fork_2,Fork_3,Fork_4,Fork_5
0.0,1.0,1.0,1.0,1.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,1.0,1.0,1.0
0.019080886690695926,1.0,0.0,1.0,1.0,1.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,0.0,1.0,1.0
0.3214129995092461,1.0,0.0,1.0,1.0,0.0,0.0,1.0,0.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,0.0,1.0,0.0
0.3520503972531178,1.0,0.0,0.0,1.0,0.0,0.0,1.0,1.0,0.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0
0.49877917097530133,1.0,0.0,0.0,0.0,0.0,0.0,1.0,1.0,1.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,1.0,0.0,0.0,0.0
0.9741824930356797,0.0,0.0,0.0,0.0,0.0,1.0,1.0,1.0,1.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0
```

Рисунок 3.3: Данные моделирования

3.3.2 Файлы данных

```
wanch@WANCHINGA:~/work/study/2026-1/2026-1==study-simulation-modeling/labs/lab05/project$ ls -la data
total 36
drwxr-xr-x 5 wanch wanch 4096 Sep  2 06:17 .
drwxr-xr-x 9 wanch wanch 4096 Sep  2 05:35 ..
-rw-r--r-- 1 wanch wanch 9584 Sep  2 06:17 dining_arbiter.csv
-rw-r--r-- 1 wanch wanch 737 Sep  2 06:17 dining_classic.csv
drwxr-xr-x 2 wanch wanch 4096 Sep  2 04:44 exp_pro
drwxr-xr-x 2 wanch wanch 4096 Sep  2 04:44 exp_raw
drwxr-xr-x 2 wanch wanch 4096 Sep  2 04:44 sims
```

Рисунок 3.4: Результаты моделирования

3.3.3 Сравнение классической сети и сети с арбитром

[illegible]

Рисунок 3.5: Сравнение данных

4 Выводы

В ходе лабораторной работы была реализована сеть Петри для задачи «Обедающие философы». Классическая модель демонстрирует возникновение взаимной блокировки (deadlock). Модифицированная модель с арбитром успешно предотвращает deadlock. Результаты подтверждают теоретические предсказания.